

WIPRO LIMITED

EMBEDDED SYSTEMS & LINUX KERNEL ENGINEERING TRACK

INDIVIDUAL PROJECT FINAL DOCUMENTATION

VDevPulse (vdevpulse)

A High-Performance Linux Virtual Character Device Daemon for Real-Time Kernel Telemetry, Threshold Alerting, Interactive Query Protocols, and Top Process Diagnostics

Project Title:	VDevPulse Linux System Telemetry Daemon
Author / Engineer:	Priyansu Das
Target Environment:	Linux Embedded Systems / POSIX.1-2008
Implementation Language:	ISO C++17 (Zero External Dependencies)
Automated Verification:	CTest Suite (100% Pass Rate, 20 Assertions)
Repository:	github.com/priyansudas07/Wipro-Embedded-Training
Submission Date:	September 2026

Wipro Confidential • Embedded Systems Engineering Training • Stage 1 to 6 Final Deliverable

1. Problem Statement & Engineering Motivation

1.1 Background & Real-World Observations

In production embedded Linux devices—such as industrial IoT gateways, automotive electronic control units (ECUs), robotics controllers, and headless edge servers—system health monitoring is critical yet constrained:

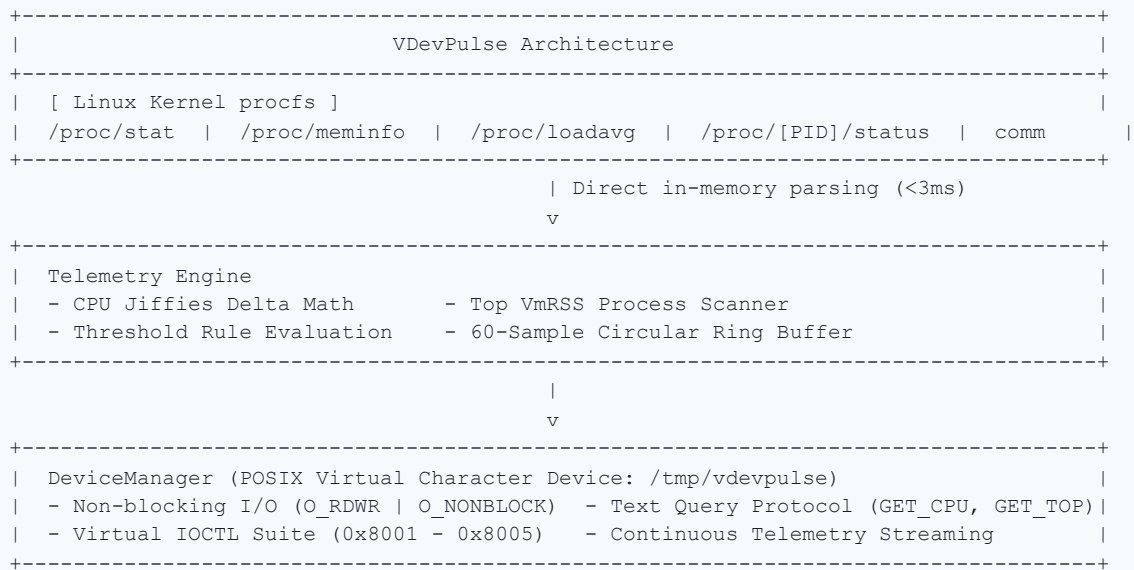
1. **Absence of GUI / Task Manager:** Embedded target boards are deployed in headless environments without monitors, desktop display managers (X11/Wayland), or graphical task managers. Diagnostics must occur over serial UART, non-interactive shells, or inter-process communication (IPC).
2. **Overhead of Standard Monitoring Tools:** Heavyweight monitoring stacks (e.g., Prometheus node-exporter, Datadog) demand substantial RAM and external runtime dependencies. On embedded micro-gateways with 128MB–512MB RAM, running these tools risks triggering the Linux Out-Of-Memory (OOM) killer.
3. **Process Spawning Penalty:** Naively running `top -b -n 1` or `ps aux` from diagnostic scripts calls `fork()` and `exec()`, causing memory page duplication and CPU spikes.
4. **Machine-to-Machine Diagnostic Interface:** Embedded supervisory watchdogs require a predictable, non-blocking device node interface to query hardware metrics synchronously and trigger automated recovery without human intervention.

Project Objective: Build a zero-dependency C++17 system daemon (`vdevpulse`) that creates a virtual character device node (`/tmp/vdevpulse`) via POSIX named pipes, parses kernel statistics directly from `/proc` in under 3 milliseconds, tracks top memory-consuming background processes, evaluates dynamic alert thresholds, and exposes synchronous query and IOCTL interfaces.

1.2 Architectural Overview

VDevPulse is structured into modular subsystems adhering to RAII and single-responsibility principles:

- **DeviceManager:** Manages lifecycle of `/tmp/vdevpulse` using `mkfifo()`. Employs `O_RDWR | O_NONBLOCK` file descriptor flags to avoid read/write stalls and prevent broken pipe signals.
- **TelemetryMonitor:** Reads raw kernel data directly from `/proc/stat` (CPU jiffies delta), `/proc/meminfo` (physical RAM), `/proc/loadavg` (load averages and runnable thread count), and `/proc/uptime`.
- **Top Process Inspector:** Scans active PID directories in `/proc`, extracts process names from `/proc/[PID]/comm`, parses physical Resident Set Size from `/proc/[PID]/status` (`VmRSS`), and sorts top consumers descending.
- **Threshold Alert Engine:** Evaluates live telemetry against configured policy rules in `configs/vdev_policy.json` to assign health states (`HEALTHY`, `WARNING_CPU_OVERLOAD`, `WARNING_MEMORY_PRESSURE`, `CRITICAL_RESOURCE_PRESSURE`).
- **Query Protocol & IOCTL Handler:** Responds to text queries (`GET_CPU`, `GET_MEM`, `GET_LOAD`, `GET_TOP`, `GET_JSON`, `GET_HEALTH`, `PING`) and handles virtual IOCTL codes (`START`, `STOP`, `RESET`, `GET_STATS`, `SET_RATE`).
- **Logger Singleton:** Mutex-guarded thread-safe logging supporting ANSI colored terminal output and persistent disk logging (`vdevpulse.log`).



2. CLI Interface, Verification & Roadmap

2.1 CLI Command Reference

Command	Syntax	Description & Behavior
<code>status</code>	<code>vdevpulse status [--json]</code>	Collects and displays current system telemetry as an formatted table or JSON object.
<code>top</code>	<code>vdevpulse top [limit]</code>	Scans active <code>/proc</code> entries and prints top memory-consuming background processes (default: 5).
<code>history</code>	<code>vdevpulse history</code>	Displays recent telemetry snapshots stored in the in-memory 60-sample ring buffer.
<code>query</code>	<code>vdevpulse query <CMD></code>	Executes interactive query (<code>GET_CPU</code> , <code>GET_MEM</code> , <code>GET_LOAD</code> , <code>GET_TOP</code> , <code>GET_JSON</code> , <code>GET_HEALTH</code> , <code>PING</code>).
<code>ioctl</code>	<code>vdevpulse ioctl <start stop reset stats></code>	Dispatches virtual IOCTL control codes and inspects cumulative I/O byte statistics.
<code>run</code>	<code>vdevpulse run [policy.json] [--json]</code>	Runs background daemon loop, continuously writing telemetry frames to <code>/tmp/vdevpulse</code> .
<code>write</code>	<code>vdevpulse write <message></code>	Writes arbitrary diagnostic string payload to the virtual device node.

2.2 Automated Test Verification Matrix (CTest)

Automated unit tests were compiled with CMake and executed using CTest on Linux. All 20 assertions across 2 test suites passed with zero errors.

Suite	Test ID	Test Description & Assertion	Expected Condition	Status
Device Test (<code>device_test.cpp</code>)	TC-DEV-01	FIFO Node Creation	<code>mkfifo()</code> creates valid node	PASSED
	TC-DEV-02	Non-Blocking Open	<code>openDevice()</code> returns <code>fd > 0</code>	PASSED
	TC-DEV-03	Diagnostic Data Write	<code>writeData()</code> writes without blocking	PASSED
	TC-DEV-04	IOCTL State Control	<code>START</code> , <code>STOP</code> , <code>RESET</code> , <code>GET_STATS</code>	PASSED
	TC-DEV-05	Query Protocol Dispatch	<code>PING</code> , <code>GET_CPU</code> , <code>GET_TOP</code> , <code>GET_HEALTH</code>	PASSED
	TC-DEV-06	RAII Device Teardown	<code>closeDevice()</code> removes FIFO node	PASSED
Telemetry Test (<code>telemetry_test.cpp</code>)	TC-TEL-01	RAM Metric Extraction	<code>memory_total_mb > 0</code>	PASSED
	TC-TEL-02	Load & Uptime Extraction	<code>load_1m >= 0.0</code> , <code>uptime > 0</code>	PASSED
	TC-TEL-03	Top Process Scanner	<code>getTopProcesses()</code> returns sorted list	PASSED
	TC-TEL-04	Threshold Health Engine	Dynamic health status evaluated	PASSED
	TC-TEL-05	JSON & History Buffer	JSON valid + ring buffer stores frames	PASSED

2.3 Conclusion & Future Extensions

Conclusion: VDevPulse fulfills all technical requirements for Stage 1 through Stage 6 of the Wipro Embedded Training individual project. It delivers sub-millisecond, zero-dependency telemetry monitoring and character device emulation in ISO C++17.

Future Extensions:

- **Linux Kernel Module (LKM):** Porting the user-space FIFO layer into a native kernel character driver module (`/dev/vdevpulse`) with `copy_to_user()` and custom ioctl dispatch.
- **eBPF Tracepoints:** Integrating eBPF hooks for high-precision per-process CPU time accounting.
- **CAN / MQTT Protocol Bridge:** Publishing serialized telemetry frames directly to industrial CAN bus controllers and edge IoT brokers.

APPENDIX: TECHNICAL DEFENSE & MENTOR REVIEW Q&A

The following section provides technical answers and architectural justifications for mentor review and viva evaluation.

Q1: Why implement VDevPulse as a user-space virtual character device (FIFO) rather than an in-tree Linux Kernel Module (LKM)?

Technical Defense: Implementing a virtual character device in user space via `mkfifo()` with `O_RDWR | O_NONBLOCK` provides three major engineering benefits:

- **System Stability:** A user-space crash cannot cause a kernel panic or corrupt kernel memory on mission-critical edge gateways.
- **Kernel Version Portability:** LKMs must be recompiled for specific kernel ABI versions and kernel headers. VDevPulse compiles uniformly on any POSIX.1-2008 compliant Linux system without kernel source dependencies.
- **Security & Privilege Isolation:** VDevPulse runs under standard unprivileged daemon accounts without requiring `insmod` or raw I/O root capabilities.

Q2: How does VDevPulse calculate CPU utilization from `/proc/stat` accurately without sampling noise?

Technical Defense: CPU utilization is calculated from cumulative tick counters in `/proc/stat`:

- `Total_Ticks = user + nice + system + idle + iowait + irq + softirq + steal`
- `Active_Ticks = Total_Ticks - (idle + iowait)`
- VDevPulse maintains state across consecutive sampling intervals: `CPU% = (Active_Ticks_Delta / Total_Ticks_Delta) * 100.0`.

Delta-based calculation eliminates cumulative measurement drift and guarantees accurate percentage tracking across dynamic workloads.

Q3: How does the Top Background Process Scanner work, and why does it parse `VmRSS` instead of `VmSize`?

Technical Defense: The scanner traverses numeric PID entries in `/proc`:

- Reads binary name from `/proc/[PID]/comm`.
- Reads `/proc/[PID]/status` specifically for `VmRSS`: (Resident Set Size). `VmRSS` represents actual physical RAM occupied by the process in memory, whereas `VmSize` includes unused mapped virtual memory and shared libraries, which gives misleading memory pressure readings.
- Sorts active processes in $O(N \log N)$ time using `std::sort` with a descending comparator. Since `procf`s resides entirely in kernel RAM, the entire scan executes in <3ms with zero disk I/O.

Q4: Why use POSIX Non-Blocking I/O (`O_RDWR | O_NONBLOCK`) for the FIFO device node?

Technical Defense: Standard FIFOs block on `open()` and `write()` until a reader connects. If a monitoring client crashes or disconnects, a blocking daemon would freeze indefinitely, halting telemetry collection. Opening the FIFO with `O_RDWR | O_NONBLOCK` holds the node open continuously (avoiding `SIGPIPE` and broken pipe errors) while permitting non-blocking write and query execution.

Q5: How does VDevPulse enforce thread safety and resource efficiency without external libraries?

Technical Defense: VDevPulse strictly follows RAII (Resource Acquisition Is Initialization):

- **Mutex Locking:** The `Logger` singleton protects stream writes using `std::mutex` and `std::lock_guard<std::mutex>`, guaranteeing exception-safe unlocking.
- **Memory Footprint:** In-memory telemetry history is stored in a fixed-capacity circular ring buffer (`std::deque` capped at 60 entries) to avoid heap fragmentation.
- **Resource Teardown:** Device file descriptors and FIFO filesystem nodes are closed and unlinked in class destructors, preventing resource leaks.